

Übung 011 – Validierung

AUSGANGSPUNKT
lauffähige API,
Validierung fehlt

ZU SCHREIBEN
zwei `TODO`-Stellen in
`Program.cs`

HILFSMITTEL
Doku ja, **AI nein**

PRÜFEN
mit `courses.http`

Worum es geht

Im Ordner `011_Validierung_Angabe` liegt die Kursverwaltung aus Übung 011 – vollständig und lauffähig, mit allen sechs Endpunkten. Nur eines fehlt: die **Validierung**. `POST /courses` und `PUT /courses/{id}` nehmen derzeit alles an – einen leeren Titel, einen negativen Preis, null Plätze, einen Kursbeginn im Jahr 2020. Die Datenbank speichert es klaglos.

Ihre Aufgabe: Schreiben Sie die Prüfungen, die verhindern, dass ungültige Kurse entstehen. Wo genau, ist in `Program.cs` mit `TODO` markiert. Alles andere bleibt unverändert.

Ohne AI. Sie dürfen die Microsoft-Dokumentation und die Präsentation `02_Minimal_Api` verwenden. Die Musterlösung im Ordner `011_Minimal_Api` sehen Sie erst an, wenn Ihre Lösung alle Anfragen in `courses.http` besteht – sie enthält übrigens absichtlich einen Fehler, den Ihre Lösung *nicht* haben darf.

Starten

```
dotnet run --project 011_Validierung_Angabe
```

Die Anwendung läuft auf `http://localhost:5080`. Probieren Sie zuerst die Anfrage „*Alles falsch*“ aus `courses.http`: Heute antwortet sie mit `201 Created`. Wenn Sie fertig sind, muss dieselbe Anfrage `400 Bad Request` liefern.

1. Die Regeln

Beide Endpunkte – `POST` und `PUT` – prüfen **dieselben** Regeln an den Daten im Request-Body:

Feld	Regel	Meldung
<code>title</code>	darf nicht leer sein (auch nicht nur aus Leerzeichen bestehen)	Titel darf nicht leer sein.
	höchstens 120 Zeichen – <i>120 sind erlaubt</i>	Titel darf höchstens 120 Zeichen haben.
	eindeutig: kein anderer Kurs hat denselben Titel	Ein Kurs mit diesem Titel existiert bereits.
<code>trainer</code>	darf nicht leer sein (auch nicht nur aus Leerzeichen bestehen)	Trainer darf nicht leer sein.
<code>priceEuro</code>	nicht negativ – <i>0 € ist erlaubt</i> (Gratiskurs)	Preis darf nicht negativ sein.
<code>seats</code>	zwischen 1 und 200 – <i>beide Grenzen sind erlaubt</i>	Plätze müssen zwischen 1 und 200 liegen.
<code>startsOn</code>	liegt nach dem heutigen Tag – <i>heute ist schon zu spät</i>	Kursbeginn muss in der Zukunft liegen.

Die Feldnamen sind die JSON-Namen, also klein geschrieben (`title`, nicht `Title`). `EnrolledCount` und `Id` werden nicht geprüft – der Client darf sie gar nicht bestimmen.

2. Wie Fehler gemeldet werden

- Sammeln Sie die Fehler in einem `Dictionary<string, string[]>`: Schlüssel ist der Feldname, Wert die Meldung(en).

- Ist mindestens eine Regel verletzt, antwortet der Endpunkt mit `return Results.ValidationProblem(errors);` – das ergibt `400 Bad Request` mit einem standardisierten Body (*ValidationProblemDetails*).
- **Alle** verletzten Felder auf einmal melden, nicht nur das erste. Wer fünf Fehler macht, soll nicht fünfmal abschicken müssen.
- **Ausnahme beim Titel:** Hier genügt die *erste* verletzte Regel, in der Reihenfolge leer → zu lang → schon vergeben. Ein leerer Titel muss nicht auch noch auf Eindeutigkeit geprüft werden – das spart eine Datenbankabfrage.
- Nur wenn `errors` leer ist, wird gespeichert.

So sieht die Antwort auf „*Alles falsch*“ aus (gekürzt):

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "title": [ "Titel darf nicht leer sein." ],
    "trainer": [ "Trainer darf nicht leer sein." ],
    "priceEuro": [ "Preis darf nicht negativ sein." ],
    "seats": [ "Plätze müssen zwischen 1 und 200 liegen." ],
    "startsOn": [ "Kursbeginn muss in der Zukunft liegen." ]
  }
}
```

3. Neuer Kurs – `POST /courses`

Die Validierung steht **am Anfang** des Endpunkts, vor allem anderen. Geprüft wird der Parameter `course`. Die Eindeutigkeit des Titels prüfen Sie mit einer Abfrage auf `db.Courses` – gibt es irgendeinen Kurs mit diesem Titel, ist er vergeben.

4. Kurs ändern – `PUT /courses/{id}`

Dieselben Regeln, mit drei Unterschieden, auf die es ankommt:

1. **Erst 404, dann 400.** Gibt es den Kurs nicht, antwortet der Endpunkt mit `404` – auch dann, wenn die Daten im Body ungültig sind. Die Reihenfolge im Code steht schon so da: Ihre Validierung kommt *nach* dem `FindAsync`.
2. **Geprüft wird `input`**, also das, was der Client schickt – nicht `course`, das ist der gespeicherte Stand aus der Datenbank.
3. **Der eigene Titel ist kein Duplikat.** Wer Kurs 1 ändert und den Titel gleich lässt, darf nicht mit „existiert bereits“ abgewiesen werden. Bei der Eindeutigkeit zählt nur ein *anderer* Kurs – einer mit einer anderen `Id`.

Und – auch wenn es selbstverständlich klingt: **alle** Regeln gelten, einschließlich des Kursbeginns. Ein Kurs darf auch durch Ändern nicht in die Vergangenheit rutschen.

5. Nicht Teil der Aufgabe

- `POST /courses/{id}/enrol` bleibt, wie er ist. „Ausgebucht“ und „hat bereits begonnen“ sind keine Eingabefehler, sondern betreffen den Zustand des Kurses – deshalb `409 Conflict`, nicht `400`.
- Dass `EnrolledCount` beim Anlegen auf 0 gesetzt wird, ist schon erledigt.
- Keine Data Annotations (`[Required]`, `[Range]` ...), kein FluentValidation, keine zusätzlichen Pakete. Die Prüfungen stehen als gewöhnlicher C#-Code im Endpunkt.

Nützliche Bausteine

Ausdruck	Wofür
<code>string.IsNullOrEmpty(s)</code>	leer, <code>null</code> oder nur Leerzeichen
<code>DateOnly.FromDateTime(DateTime.Today)</code>	das heutige Datum als <code>DateOnly</code>
<code>await db.Courses.AnyAsync(c => ...)</code>	gibt es mindestens einen Kurs, der die Bedingung erfüllt?
<code>errors["seats"] = ["..."];</code>	Fehler eintragen (Collection Expression, ergibt ein <code>string[]</code>)
<code>if (errors.Count > 0) ...</code>	wurde irgendein Fehler gefunden?

6. Prüfen

Im Ordner liegt eine `courses.http` mit 17 Anfragen. Hinter jeder steht im Kommentar, was sie liefern **muss**. Löschen Sie vorher `courses.db`, starten Sie die Anwendung und schicken Sie die Anfragen **der Reihe nach** ab – die späteren bauen auf den ersten beiden Kursen auf.

Gruppe	Was geprüft wird	Erwartet
Ausgangslage	zwei gültige Kurse, einer mit 0 € und 200 Plätzen	201 · 201
POST, je eine Regel verletzt	jede Regel einzeln, dazu der Grenzwert 120 Zeichen	400 (Grenzwert: 201)
POST, alles falsch	alle fünf Felder stehen in <code>errors</code>	400
PUT, eigener Titel	Titel unverändert lassen ist erlaubt	200
PUT, fremder Titel	Titel eines anderen Kurses übernehmen	400
PUT, Beginn 2020	die Datumsregel gilt auch beim Ändern	400
PUT, alles falsch	alle fünf Felder stehen in <code>errors</code>	400
PUT auf <code>/courses/999</code>	unbekannter Kurs mit ungültigen Daten	404

Schauen Sie bei jedem `400` auch in den Body: Steht *genau das* Feld in `errors`, das im Kommentar genannt ist – und kein anderes?

Fertig, wenn ...

- ☐ `dotnet build` ohne Fehler und ohne Warnungen durchläuft,
- ☐ alle 17 Anfragen in `courses.http` den erwarteten Statuscode liefern,
- ☐ bei jedem `400` die richtigen Feldnamen in `errors` stehen,
- ☐ außerhalb der beiden `TODO`-Stellen nichts verändert wurde.

Zum Schluss – drei Fragen

Schriftlich beantworten, je zwei bis drei Sätze. Sie werden in der nächsten Einheit besprochen.

1. **Zählen Sie.** Wie oft steht jede Regel jetzt in `Program.cs`? Morgen kommt eine neue dazu: „Der Preis pro Platz muss unter 500 € liegen.“ An wie vielen Stellen müssen Sie sie hinschreiben – und was passiert, wenn Sie eine vergessen?
2. **Vergleichen Sie** Ihre Lösung jetzt mit `011_Minimal_Api/Program.cs`. Welche Prüfung fehlt dort im `PUT`? Welche Anfrage in `courses.http` hätte den Fehler aufgedeckt? Wie ist er vermutlich entstanden?
3. **Ohne HTTP.** Eine Blazor-Seite soll Kurse mit denselben Regeln anlegen. Können Sie Ihre Validierung von dort aus aufrufen? Was hindert Sie daran?

Zusatz für Schnelle: Lagern Sie die Prüfungen in eine einzige lokale Funktion aus, die `POST` und `PUT` gemeinsam verwenden. Was wird dadurch besser – und welches Problem aus Frage 3 bleibt trotzdem bestehen?